

Linear $P(k)$ emulators at $z = 0$ for DES halo-model pipelines (v2c)

J. Esteves

April 2026

1 Introduction

The halo-model pipeline used in DES cluster analyses (`y3_cluster_cpp`, `y1_mock_emcee.ini`) requires two linear matter power spectra: the total matter power spectrum $P^{\text{mm}}(k)$ (includes massive neutrinos) and the CDM+baryon power spectrum $P^{\text{cb}}(k)$, the latter consumed by `mf_tinker` when `matter_power_lin_version = 2`. A full CAMB evaluation costs several seconds per cosmology, which dominates the wall time of any sampling pipeline. We emulate both spectra at $z = 0$ with COSMOPOWER neural networks and recover the redshift dependence at inference time through the growth factor, $P(k, z) = D(z)^2 P(k, 0)$, evaluated by the CSL `growth_factor` module. Non-linear quantities ($\sigma(R)$, $\xi_{\text{NL}}(r)$, halo bias $b(M)$) are reconstructed at inference from the emulated spectra via `cluster_toolkit`; only the truly non-separable non-linear $P(k, z)$ requires a follow-up emulator, which is discussed in Section 5.

The emulators described here (version v2c) are trained on a combined dataset of 2×10^5 Latin-Hypercube cosmologies: an initial broad-prior LHS (v2, 10^5 samples) augmented with a Planck-dense LHS (10^5 samples) to improve accuracy in the region where the BAO feature is most sensitive.

2 Training data and prior

Priors. Two Latin-Hypercube samples were generated. The first (v2) uses broad, flat priors over the six cosmological parameters. The second (v2_planck) tightens Ω_m , n_s , and $\log(10^{10} A_s)$ to Planck 2018 $\pm 10\sigma$ and broadens Ω_b from v2's overly-tight $[0.05, 0.06]$ to a Planck-consistent range. Table 1 summarises both.

To avoid unphysical configurations with $\Omega_b > \Omega_m$ we sample Ω_{cdm} directly and set $\Omega_m = \Omega_{\text{cdm}} + \Omega_b$.

CAMB configuration. Each sample is evaluated at $z = 0$ with the CSL CAMB interface. `power_spectra = delta_tot delta_nonu` returns both $P^{\text{mm}}(k)$ and $P^{\text{cb}}(k)$ in a single call. Spectra are stored on a log-spaced k -grid with $k_{\text{min}} = 10^{-4} h/\text{Mpc}$, $k_{\text{max}} = 50 h/\text{Mpc}$, and $n_k = 506$ modes, for a row of 512 floats (six parameters + 506 $P(k)$ values) per cosmology per spectrum.

Dataset. CAMB returned 99 939 valid rows from the v2 LHS and 99 995 from the v2_planck LHS, for a combined total of $N_{\text{tot}} = 199 934$ cosmologies after integrity alignment (linear and CDM+baryon row counts matched per-cosmology; 0.03% lost to transient filesystem errors). The combined dataset is shuffled and split 90/10 into 179 940 training and 19 994 test samples, so the test set covers both broad- and Planck-dense regions.

Parameter	Symbol	v2 range	v2_planck range ($\pm 10\sigma$)
Hubble	h_0	[0.40, 1.00]	[0.40, 1.00]
CDM density	Ω_{cdm}	[0.02, 0.65]	[0.182, 0.338]
Baryon density	Ω_b	[0.050, 0.060]	[0.0376, 0.0608]
Scalar tilt	n_s	[0.87, 1.07]	[0.923, 1.007]
Amplitude	$\ln(10^{10} A_s)$	[1.64, 4.44]	[2.904, 3.184]
Neutrino mass	$\sum m_\nu$ [eV]	[0.00, 0.20]	[0.00, 0.20]
(Derived)	Ω_m	[0.070, 0.710]	[0.219, 0.411]

Table 1: Cosmological priors for the two LHS components of the v2c training set. The v2 box is broad and uniform. The v2_planck box is centred on Planck 2018 best fits and tightened to $\pm 10\sigma$ on the three parameters that drive the linear $P(k)$ accuracy (Ω_m , n_s , $\ln(10^{10} A_s)$), plus a Planck-compatible Ω_b range derived from $\Omega_b h^2 = 0.02208 \pm 0.00052$. Dark energy is fixed to a Λ CDM prescription ($w = -1$, $w_a = 0$, $\Omega_k = 0$) with three massive neutrinos in a normal hierarchy and $N_{\text{eff}} = 3.046$.

3 Emulator architecture and accuracy

Each emulator is a COSMOPOWER fully-connected network with four hidden layers of 512 units and custom sigmoid-based activations, mapping the six-dimensional parameter vector to $\log_{10} P(k)$ sampled on the 506-mode grid. We train on $\log_{10} P(k)$ because $P(k)$ spans ~ 5 decades over the prior; the network regresses log-linear features more efficiently. The two emulators share architecture but are trained independently on $P^{\text{mm}}(k)$ and $P^{\text{cb}}(k)$.

Training runs three phases of decreasing learning rate and increasing batch size: (10^{-3} , 10^{-4} , 10^{-5}) and (10^3 , 5×10^3 , 10^4) respectively, for (400, 800, 1200) maximum epochs with a patience of 30 epochs on the validation loss. An 80/20 internal train/validation split is used within the 179 940 training samples.

Accuracy is reported in terms of the fractional residual

$$\epsilon(k) \equiv \frac{P_{\text{pred}}(k)}{P_{\text{true}}(k)} - 1,$$

evaluated on every $(k, \text{cosmology})$ pair in the 19 994-sample test set ($\sim 10^7$ predictions per emulator). Table 2 summarises the test-set performance. The combined v2c dataset roughly doubles the training volume over v2 and densifies the Planck region, which drives a ~ 25 – 35% reduction in every quantile of $|\epsilon|$ relative to the v2-only baseline.

Figure 1 shows the cumulative distribution of $|\epsilon|$ across all test points; both emulators reach 98% of predictions below 1%. Figure 2 shows the k -dependence of the median and 95th percentile; the residual oscillation near the BAO scale $k \sim 0.1$ – $0.3 h/\text{Mpc}$ that limited v2 is strongly suppressed by the Planck-dense samples. Figure 3 compares emulated to true spectra on four test cosmologies spanning the edges of the Ω_m and m_ν priors.

4 Compute pipeline

Both CAMB LHS runs were generated on the Perlmutter CPU partition as SLURM job arrays. The conda-bundled `mpi4py` in the shared cosmosis environment does not link to Cray MPICH, so an MPI launch fails silently with every rank reporting `Get_size() = 1`. We work around this

Quantity	$P^{\text{mm}}(k)$		$P^{\text{cb}}(k)$	
	v2	v2c	v2	v2c
Training samples	89 945	179 940	89 945	179 940
Test samples	9 994	19 994	9 994	19 994
RMSE $[\log_{10} P(k)]$	0.0144	0.0104	0.0143	0.0104
Median $ \epsilon $	0.089%	0.068%	0.094%	0.076%
95th percentile $ \epsilon $	0.70%	0.46%	0.75%	0.54%
99th percentile $ \epsilon $	2.01%	1.27%	1.95%	1.49%
Fraction $ \epsilon < 1\%$	96.9%	98.5%	96.8%	98.1%
Fraction $ \epsilon < 5\%$	99.7%	99.9%	99.7%	99.8%

Table 2: Test-set accuracy with $\epsilon = P_{\text{pred}}/P_{\text{true}} - 1$ per $(k, \text{cosmology})$, comparing v2 (broad prior only) to v2c (broad + Planck-dense). The v2c test set is drawn from the shuffled combined dataset; it contains both broad-prior and Planck-dense cosmologies and is therefore not directly comparable to the v2 test set row-by-row, but it is the honest benchmark of the v2c emulator.

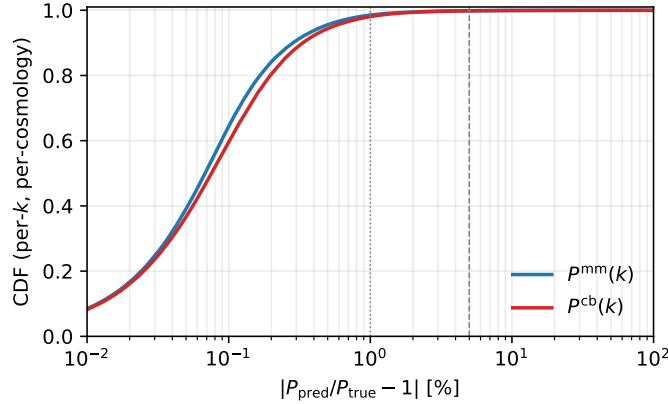


Figure 1: Cumulative distribution of $|\epsilon|$ across all $19\,994 \times 506$ test points. Dotted line: 1%; dashed line: 5%.

by splitting each 10^5 -row LHS into 100 slices of 1000 samples and running one serial cosmosis invocation per array task, with each task writing to a distinct `slice{ID}[planck_]` file prefix to avoid concurrent writes on the same inode.

Table 3 lists the per-step SLURM parameters. The `shared` qos is ideal for the CAMB arrays because the 0.5-node cap is not restrictive for a single-core task and its higher concurrency limit ($\gg$ `debug`’s `MaxJobsPU=2`) lets us run 20 tasks in parallel; we throttle with `%20` to avoid dominating the shared scheduler. The Planck-dense run is noticeably slower per sample than the broad v2 run: CAMB converges its internal k and z integrations more slowly for cosmologies close to the physically motivated best-fit point because the halofit and transfer caches demand tighter tolerances. This is irrelevant for production — the emulator is used at inference and completes in milliseconds — but it explains why the 2 h wall-time budget was nearly saturated by the slowest Planck tasks.

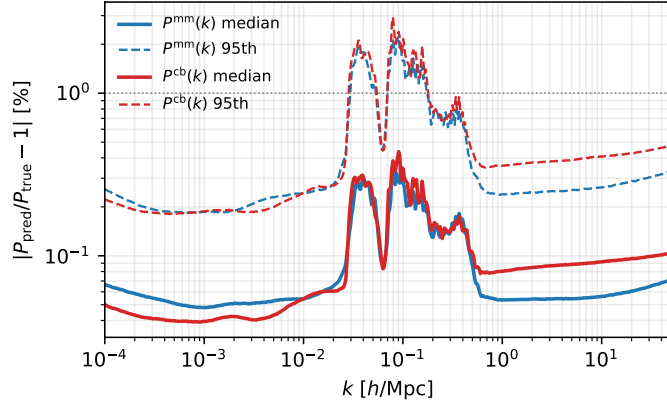


Figure 2: Median (solid) and 95th percentile (dashed) of $|\epsilon|$ as a function of k . Both emulators stay below 1% on the median across the entire k -range.

Step	QoS	Nodes $\times$ CPU/task	Array	Wall/task	Notes
v2 CAMB	shared	1×1	0–99, %20	~ 45 min	$100 \times 1\,000$
v2_planck CAMB	shared	1×1	0–99, %20	~ 60 – 105 min	$100 \times 1\,000$
Merge + split	shared	1×1	—	~ 3 min	streaming concat
Clean (.dat $\rightarrow$.npy)	debug	1×32	—	30 s	polars CSV parse
Emulator training	gpu_debug	1×32 (1 GPU)	—	~ 7 min	A100-80GB, from scratch

Table 3: SLURM layout of the v2c pipeline. Total CAMB cost dominates at ~ 150 CPU-hours per LHS ($100 \times 1 \times 1.5$ h average for the Planck-dense run). End-to-end wall time from LHS generation to trained emulators was ~ 10 hours including queue waits and the slower Planck-dense wall times.

5 Next steps: non-linear $P(k, z)$

The linear pipeline is separable in z , which is why the v2c emulators are trained at a single redshift. The non-linear pipeline is not: the halofit prescription introduces a non-trivial z -dependence beyond the linear growth factor, so an emulator of the non-linear boost $B(k, z) \equiv P_{\text{nl}}(k, z)/P_{\text{lin}}(k, z)$ must take z as an explicit input.

Planned setup. Reuse the six-parameter Planck-dense LHS from v2c and add z as a seventh input axis, either by broadcasting each cosmology across an n_z -point redshift grid or by drawing an additional LHS dimension over z . The downstream DES pipeline samples $z \in [0.1, 0.8]$, so a training grid of $n_z = 11$ log-spaced redshifts over $[0, 2]$ (extra margin for σ_R integrals and 2-halo integrals out to $z \approx 2$) is sufficient. The CAMB configuration becomes `power_spectra = delta_tot delta_nonu` with `nonlinear = both`, giving linear and non-linear spectra in a single call; the boost is $B = P_{\text{nl}}/P_{\text{lin}}$ computed in the CosmoSIS save module.

Cost estimate. With $n_z = 11$, each CAMB call is $\sim 3\times$ slower than the $z = 0$ case because halofit does internal k - z integrations at every output z . 100 slices $\times$ 1000 samples at ~ 15 s per sample gives ~ 250 CPU-hours, plus the same merge/clean/training overhead. The training network gains a seventh input. Because the boost varies over a smaller dynamic range (~ 0.1 – 10), the network typically converges in fewer epochs.

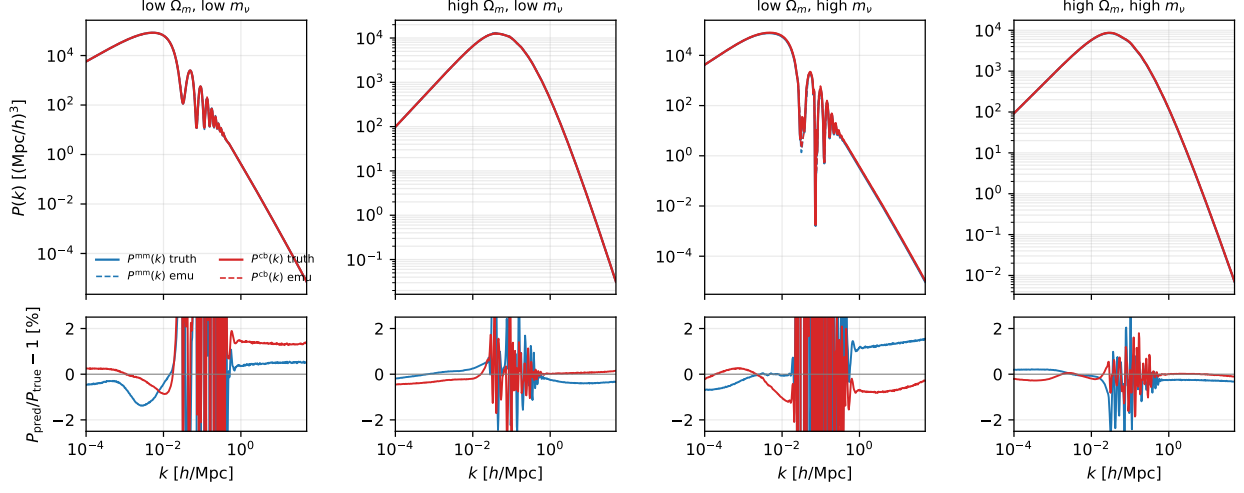


Figure 3: Four held-out test cosmologies spanning the corners of the (Ω_m, m_ν) prior. Top: true (solid) and emulated (dashed) $P(k)$ for both spectra. Bottom: fractional residual $\epsilon = P_{\text{pred}}/P_{\text{true}} - 1$ in percent.

Derived halo-model quantities. Once both the linear emulators and the non-linear boost emulator are in place, every halo-model quantity used by `y3.cluster.cpp` is recoverable at inference: $\sigma(R, z)$ from the CSL `sigma.cpp` on $P_{\text{lin}}(k, z)$; dn/dM from `mf.tinker` on $P_{\text{lin}}^{\text{cb}}$; $b(M)$ from `cluster_toolkit.peak_height.nu_at_M` on the same; and $\xi_{\text{NL}}(r, z)$ from `cluster_toolkit.xi.xi_mm_at_r` on $P_{\text{nl}} = B \cdot P_{\text{lin}}$. No further emulators are required for the current cluster-cosmology programme. A follow-up direct emulator of $\sigma(M, z)$ would further reduce the inference-time bottleneck in `mf.tinker`, but this is a second-order optimisation rather than a necessity.

Validation plan. The accuracy targets for the non-linear pipeline flow from the 1% theoretical error budget on the Tinker mass function. We will demand the boost satisfy $|\epsilon_B| < 1\%$ at the median and $< 2\%$ at the 95th percentile, and track the propagated error on $b(M)$ (target: $< 1\%$ at fixed ν) and $\xi_{\text{NL}}(r)$ (target: $< 2\%$ at $r < 100 \text{ Mpc}/h$) on a held-out set of cosmologies.